

Server Implementation & Infrastructure Requirements

Self-Hosted Jitsi Meet & Jibri Screen Recording (Migration from 8x8 JaaS)
Scenario: 1 Host (Screen Share + Audio) + 500 Attendees | Output: 1080p MP4

1. Target Workload & Traffic Overview

This specification outlines the server hardware, networking, kernel tuning, and configuration required to deploy a self-hosted Jitsi Meet cluster with dedicated Jibri server-side recording, replacing 8x8 JaaS on your dedicated platform.

- 1 Host (Presenter):** Streams 1080p screen share (~3.0 Mbps), webcam (~1.5 Mbps), and audio (~48 kbps). Ingress to server is ~4.5 Mbps.
- 500 Attendees (Viewers):** Connect in listen-only mode with camera and microphone muted. Each attendee receives the 1080p presentation stream. Outbound WebRTC egress from JVB peaks at 1.25 Gbps to 1.90 Gbps.
- Jibri Recorder:** Runs as a headless containerized participant with channelLastN=1 (ignoring the 500 attendees). It auto-pins the host screen share, capturing clean 1080p 30fps video and PulseAudio sound into an MP4 file.

2. Server Hardware Specifications

Because 500 concurrent participants downloading a 1080p screen share generate between 1.25 Gbps and 1.90 Gbps of outbound WebRTC UDP traffic, the server hosting Jitsi Videobridge (JVB) requires a dedicated 10 Gbps network interface.

Option A: Recommended Single Bare-Metal Node (All-in-One)

All services (Nginx, Prosody, Jicofo, JVB, and Jibri) run on one high-performance bare-metal host (e.g., Hetzner AX102, OVH Advance, or AWS bare-metal).

Hardware Component	Minimum Specification	Technical Purpose / Rationale
Processor (CPU)	16 Cores / 32 Threads (AMD EPYC, Ryzen 9, Intel Xeon)	Handles JVB WebRTC packet routing, Chromium headless rendering, and real-time x264 FFmpeg encoding.
System RAM	64 GB DDR4/DDR5 ECC	Buffers 500 WebRTC peer connections and isolates Chromium

		browser execution.
Network Card (NIC)	10 Gbps Port (Dedicated/Unmetered)	CRITICAL: Outbound WebRTC traffic to 500 users is 1.3 - 2.0 Gbps. Standard 1 Gbps ports will bottleneck.
Storage (OS & Apps)	500 GB NVMe SSD (PCIe Gen4)	Fast system boot, container images, and temporary cache.
Storage (Recordings)	1 TB NVMe SSD (Dedicated mount)	Dedicated sequential write buffer for 1080p MP4 recordings (~1.85 GB/hour).
Operating System	Ubuntu Server 22.04 or 24.04 LTS	Modern WebRTC socket support and current Docker Compose V2 runtime.

Option B: Distributed 3-Node Cluster

Splits the signaling, media routing, and video encoding across three specialized servers:

Node Role	vCPU / RAM	Network Port	Assigned Workload
Node 1: Control Plane	4 vCPU / 8 GB RAM	1 Gbps NIC	Nginx Web SSL, Prosody XMPP (JWT Auth), and Jicofo.
Node 2: Media Bridge (JVB)	16 vCPU / 32 GB RAM	10 Gbps NIC (Crucial)	Routes WebRTC audio/video/screen packets to 500 users.
Node 3: Recording (Jibri)	8 vCPU / 16 GB RAM	1 Gbps NIC + 1TB NVMe	Runs headless Chrome, PulseAudio, and FFmpeg encoder.

3. Container Resource Limits & Docker Shared Memory

To ensure system stability, configure CPU and RAM boundaries in docker-compose.yml and jibri.yml. Note that Jibri requires shm_size: '4gb' to prevent Chromium from crashing during 1080p decoding.

```
services:
  jvb:
    deploy:
      resources:
        limits:
```

```

    cpus: '12.0'
    memory: 16384M
  reservations:
    cpus: '4.0'
    memory: 4096M

jibri:
  shm_size: '4gb' # CRITICAL: Chromium crashes without at least 2GB-4GB shared memory
  deploy:
    resources:
      limits:
        cpus: '4.0'
        memory: 8192M
      reservations:
        cpus: '2.0'
        memory: 4096M

```

4. Network Firewall & Port Matrix

Port	Protocol	Accessibility	Purpose
80	TCP	Public (0.0.0.0/0)	HTTP Web redirection & Let's Encrypt renewal
443	TCP	Public (0.0.0.0/0)	HTTPS Web client, WebSockets, and BOSH
10000	UDP	Public (0.0.0.0/0)	WebRTC audio/video media streams via JVB (Mandatory)
8443	TCP	Internal / Docker	Internal Nginx reverse proxy loopback
5222	TCP	Internal / Docker	Prosody XMPP client signaling
5347	TCP	Internal / Docker	Prosody XMPP component protocol
2222	TCP	Internal / Docker	Jibri REST health check daemon

5. Host Linux Kernel & Socket Buffer Tuning

A standard Linux kernel drops UDP packets when routing media to >100 participants. Deploy the following configuration to /etc/sysctl.d/99-jitsi-500users.conf and apply with 'sudo sysctl --system':

```
# Socket receive & send buffer maximums for WebRTC
net.core.rmem_max = 67108864
net.core.wmem_max = 67108864
net.core.rmem_default = 33554432
net.core.wmem_default = 33554432

# Buffer memory allocation for UDP queues
net.ipv4.udp_mem = 65536 131072 262144

# System-wide file descriptor limit for 500+ WebRTC peer sockets
fs.file-max = 2097152

# Ephemeral port range expansion
net.ipv4.ip_local_port_range = 1024 65535

# Backlog queue limits
net.core.netdev_max_backlog = 100000
net.core.somaxconn = 65535
```

In /etc/security/limits.conf, expand open file limits:

```
* soft nofile 65535
* hard nofile 65535
root soft nofile 65535
root hard nofile 65535
```

6. Step-by-Step Server Setup Runbook

Step 1: Install Docker & Docker Compose V2

Execute the following commands on the Ubuntu/Debian server:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl gnupg lsb-release apt-transport-https ca-certificates
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-
plugin
sudo systemctl enable --now docker
```

Step 2: Configure Environment Variables (jitsi/.env)

Configure the public domain, ports, JWT authentication, and Jibri settings in jitsi/.env:

```
PUBLIC_URL=https://meet.yourplatform.com
HTTP_PORT=80
HTTPS_PORT=443
JVB_PORT=10000

COMPOSE_FILE=docker-compose.yml:jibri.yml
JIBRI_RECORDING_DIR=/storage/recordings

ENABLE_RECORDING=1
ENABLE_SERVICE_RECORDING=1
JIBRI_BREWERY_MUC=jibribrewery

JIBRI_RECORDER_USER=recorder
JIBRI_RECORDER_PASSWORD=strong_recorder_password
JIBRI_XMPP_USER=jibri
JIBRI_XMPP_PASSWORD=strong_jibri_password

# JWT Authentication matching your platform backend
ENABLE_AUTH=1
AUTH_TYPE=jwt
JWT_APP_ID=your_platform_app_id
JWT_APP_SECRET=your_platform_app_secret
JWT_ACCEPTED_ISSUERS=your_platform_backend
JWT_ACCEPTED_AUDIENCES=jitsi

CHROMIUM_FLAGS=--autoplay-policy=no-user-gesture-required,--use-fake-ui-for-media-stream,--start-maximized,--kiosk,--enabled,--host-resolver-rules=MAP meet.yourplatform.com web:8443
```

Step 3: Configure Jibri for Host Screen Recording Only

To protect Jibri from 500 audience streams and guarantee clean slide capture, Jibri joins with conference URL parameters:

```
// Stream isolation: requests only 1 video stream (the host screen)
config.channelLastN = 1;
config.disableTileView = true;
config.disableAutoPinOnScreenShare = false;

// Clean broadcast: hides toolbars, popups, and badges from MP4
config.hideConferenceSubject = true;
config.hideConferenceTimer = true;
config.disableNotifications = true;
config.disableChat = true;
interfaceConfig.TOOLBAR_BUTTONS = [];
interfaceConfig.SHOW_JITSI_WATERMARK = false;
```

Step 4: Update Dedicated Platform Frontend (Replacing JaaS)

In your platform web application, update the Jitsi IFrame / React initialization:

```
const domain = "meet.yourplatform.com";
const options = {
  roomName: "webinar-hall-500",
  width: "100%",
  height: "100%",
  parentNode: document.querySelector("#jitsi-container"),
  jwt: clientBackendGeneratedJWT,
  configOverwrite: {
    startWithAudioMuted: !isHost, // 500 attendees start muted
    startWithVideoMuted: !isHost, // 500 attendees start with camera off
    channelLastN: 2,              // Viewers receive screen share + presenter cam
    disableAutoPinOnScreenShare: false
  },
  interfaceConfigOverwrite: {
    TOOLBAR_BUTTONS: isHost
      ? ['microphone', 'camera', 'desktop', 'recording', 'chat', 'participants-pane',
        'hangup']
      : ['chat', 'raisehand', 'hangup']
  }
};
const api = new JitsiMeetExternalAPI(domain, options);
```

Step 5: Backend JWT Token Generation (Host vs. Viewers)

```
// Host Presenter Token Payload:
{
  "context": {
    "user": { "name": "Host Presenter", "moderator": true },
    "features": { "recording": true }
  },
  "aud": "jitsi",
  "iss": "your_platform_backend",
  "sub": "meet.yourplatform.com",
  "room": "webinar-hall-500"
}

// 500 Attendees Token Payload:
{
  "context": {
    "user": { "name": "Attendee", "moderator": false },
    "features": { "recording": false }
  },
  "aud": "jitsi",
  "iss": "your_platform_backend",
  "sub": "meet.yourplatform.com",
  "room": "webinar-hall-500"
}
```

Step 6: Start Services & Health Checks

```
cd jitsi

# 1. Start all containers in background
docker compose up -d

# 2. Check running status (web, prosody, jicofo, jvb, jibri)
docker compose ps

# 3. Check Jibri health
docker exec jitsi-jibri-1 curl -s http://127.0.0.1:2222/jibri/api/v1.0/health
# Response: {"busyStatus":"IDLE","health":{"healthStatus":"HEALTHY"}}

# 4. Follow live recording logs when host starts recording
docker compose logs -f jibri
```